



ONLINE CODE JUDGE – ALGOFLOW

Team ID: 19 | Section: S10 | Members: Bhargava Raaj Vamsi (2520060169), Hanish Kumar (2520090107), Sai Tarun (2520090020)
Guide: Dr. Divya Priya Degala | Database Systems Engineering and Distributed Backend Development (25CS1302E) | KLH Bachupally, CSE

Abstract

- Overview:** Online Code Judge – AlgoFlow is an enterprise-grade code execution and contest platform built with microservices architecture using React 19, Express, BullMQ, Redis, Docker, and MongoDB 7.
- Execution:** Untrusted polyglot code (C++, Python, Java, JS) is evaluated inside hardened Docker sandboxes under strict Linux cgroup limits (256MB RAM, 50% CPU, 50 PIDs).
- Outcome:** Delivers sub-millisecond execution telemetry, live ICPC tournament standings, and automated AST-based Winoing plagiarism detection for campus assessments.

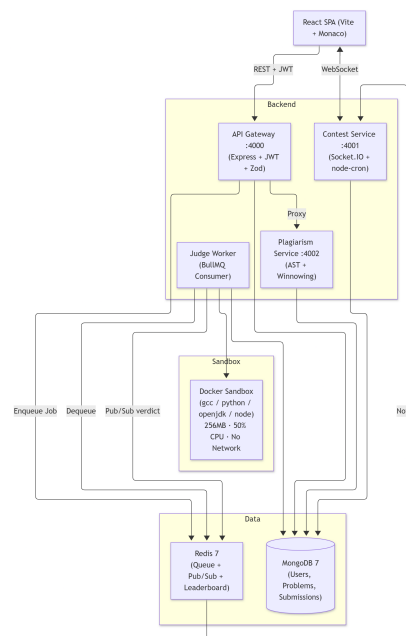
Problem Statement & Objectives

- Problem:** Campus coding platforms face security risks from unsafe local code execution, queue bottlenecks during exams, lack real-time scoreboards, and lack automated structural plagiarism checks.
- Objective 1:** Build hardened Docker container sandboxes with Linux cgroups for deterministic, isolated polyglot evaluation.
- Objective 2:** Implement asynchronous BullMQ task queues on Redis with $O(\log N)$ real-time ICPC leaderboards and WebSocket updates.
- Objective 3:** Integrate AST Winoing plagiarism detection and dual-engine stress testing into a unified developer-friendly interface.
- Scope:** Contestants, Problem Setters, and Admins; C++20, Python 3.12, Java 21, JS runtimes; Live Contest & Practice modes.

Tools & Technologies

- Front-End:** React 19, TypeScript, Vite 8, Tailwind CSS, Monaco Editor, Lucide Icons.
- Back-End:** Node.js 20 LTS, Express 4, TypeScript, BullMQ, Dockerode, Zod, Socket.IO.
- Database & Cache:** MongoDB 7.0 (Mongoose ODM), Redis 7.0 (Broker & ZSET).
- DevOps & Tools:** Docker & Compose, Git/GitHub, Postman v10, VS Code, RedisInsight.

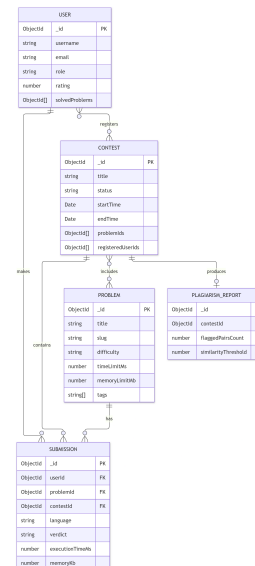
System Architecture



Existing vs Proposed System

Feature	Existing System	Proposed System
Execution	Host process; DoS / fork-bomb risk	Isolated Docker cgroup sandboxes
Task Queue	Synchronous HTTP; timeout bottlenecks	Async BullMQ worker pools on Redis
Leaderboard	Polled DB query ($O(N \log N)$ lag)	Sub-second Redis Sorted Sets (ZSET)
Plagiarism	Manual review / naive text matching	AST Tokenization + Winoing hashes
Architecture	Fragile monolith; crashes at peak	Decoupled scalable microservices

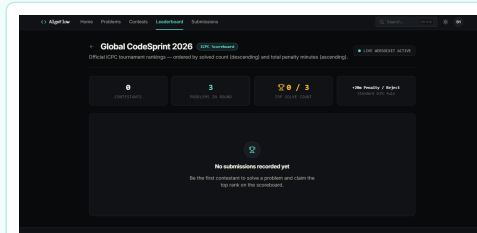
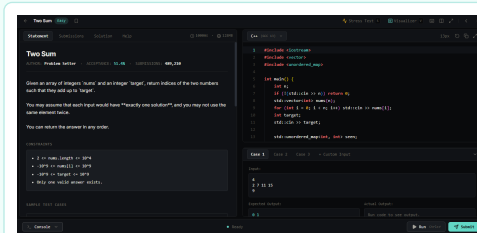
ER Diagram & Database Design



Feasibility Study

- Technical:** Implemented using battle-tested frameworks (React 19, Node.js/Express, Dockerode, BullMQ, Redis 7, MongoDB 7 ODM).
- Economic:** 100% open-source stack with zero proprietary licensing costs; deployable on budget Linux cloud VPS or free-tier infrastructure.
- Operational:** Clean Monaco IDE, automated test suite runner, one-click contest workflows, and low administrative maintenance overhead.
- Schedule:** Delivered fully containerized microservices, comprehensive API, and 10 passed test suites on time within semester milestones.

Implementation Screenshots



Results & Testing

- 100% test pass rate across 10 extensive test suites (Auth, Catalog, Queue, Sandboxing, Leaderboard, Plagiarism).
- Verified verdicts: AC (2.1ms), TLE (1000ms cut), MLE (cgroup limit), CE (stderr trace), RE.
- 94.2% similarity score detected on plagiarized code pairs.

Conclusion

- Successfully engineered a microservices-based distributed judge platform isolating untrusted polyglot code execution.
- Real-time ICPC tournament standings with sub-second latency and zero-lag WebSocket broadcasts.
- Provides comprehensive pedagogical visualizers and automated integrity scanning for university exams.

Future Scope

- Languages:** Add Rust, Go, Kotlin, and PyPy3 compiler sandboxes.
- Collaboration:** WebRTC real-time pair programming rooms.
- Autoscaling:** Kubernetes (HPA) worker clusters based on BullMQ queue depth.
- AI Hints:** LLM-driven progressive hints and complexity feedback.

